

#lang prompt-pattern

A chrestomathy member for prompt engineering, in Racket

Vivekananda Reddy Godala

SLE 2026 — Metaprogramming Assignment

University of Koblenz · Prof. Ralf Lämmel

2026-06-15

Research context

Prompt engineering is becoming a real DSL domain.

Two notable recent approaches:

Work	Style	Implementation
Impromptu — Clariso & Cabot, MODELS 2023	external textual DSL	Langium (grammar workbench)
DSPy — Khattab et al., 2023	embedded programming model	Python (signatures, modules, teleprompters)

Both treat prompts as first-class artifacts — but they sit at **opposite ends of the metaprogramming spectrum**: a stand-alone grammar workbench vs. a library embedded in a general-purpose language.

References: doi:10.1109/MODELS58315.2023.00020 · arXiv:2310.03714

The methodological frame: MetaLib

Schauss, Lämmel, Härtel, Heinz, Klein, Härtel, Berger.

A Chrestomathy of DSL Implementations. SLE 2017.

Implement the same DSL (FSML) across many metaprogramming approaches; compare them through one shared feature model.

MetaLib has members in **Java · Python · Haskell QQ · Scala · Rascal · MPS · Spoofax · EMF/Xtext · Racket · ...**

The gap I target: in the prompt-engineering space the two ends are taken (Impromptu, DSPy), but there is no **hosted-language** member — even though Racket is already a MetaLib member for FSML.

Reference: doi:10.1145/3136014.3136038

Research question and method

RQ. What feature coverage does a Racket `#lang` realize for prompt engineering, and **where in the language lifecycle** does it catch errors compared to Impromptu's Langium pipeline and DSPy's runtime modules?

Method. Implement a small `#lang prompt-pattern` in Racket and document its coverage in the MetaLib vocabulary — abstract syntax, concrete syntax, static semantics, translation (staging), dynamic semantics.

Scope — deliberately tiny: one macro, one runtime function, ~60 lines of language code. Four example programs. Three unit tests.

The language

```
#lang prompt-pattern

(define-pattern support-bot
  #:slots (question company)
  (system "You are a support agent for {company}. Be concise.")
  (user "How do I get a refund?")
  (assistant "Go to Settings -> Billing -> Refund.")
  (user "{question}"))

(render support-bot
  #:company "Acme"
  #:question "Where is my invoice?")
```

`render` returns a list of `{role, content}` records — chat-completion shape, ready for an LLM API.

The two metaprogramming moves

1. `#lang prompt-pattern` — a 2-line module reader.

```
#lang s-exp syntax/module-reader  
prompt-pattern
```

Reuses Racket's s-expression reader; swaps in our language bindings.

2. `define-pattern` — a `syntax-parse` macro that does real work at **expansion time**: read `#:slots`, scan each template for `{slot}` references, and `raise-syntax-error` on any undeclared reference — otherwise expand to a runtime value.

`define-pattern` is a two-stage program

A named lecture concept — **multi-stage programming** — is the heart of it.

Stage 1 — expansion (compile) time: check slots, build the AST.

```
(define-pattern greeting #:slots (name tone)
  (system "Your tone is {tone}.")
  (user "Say hello to {name}."))
```

expands to a plain definition of a runtime value:

```
(define greeting
  (pattern 'greeting '(name tone)
    (list (message 'system "Your tone is {tone}.")
          (message 'user "Say hello to {name}.")))))
```

Stage 2 — run time: `render` fills the templates. An ill-formed prompt is rejected in stage 1 — *before stage 2 ever exists*.

Live demo — greeting

```
#lang prompt-pattern
(define-pattern greeting
  #:slots (name tone)
  (system "You are an assistant. Your tone is {tone}.")
  (user "Say hello to {name}.")

(render greeting #:name "Vivek" #:tone "warm")
```

Real output (`make demo`):

```
[system] You are an assistant. Your tone is warm.
[user]   Say hello to Vivek.
```


Live demo — the headline insight

```
#lang prompt-pattern
(define-pattern bad
  #:slots (name)
  (user "Hello {name}, your account balance is {missing}."))
```

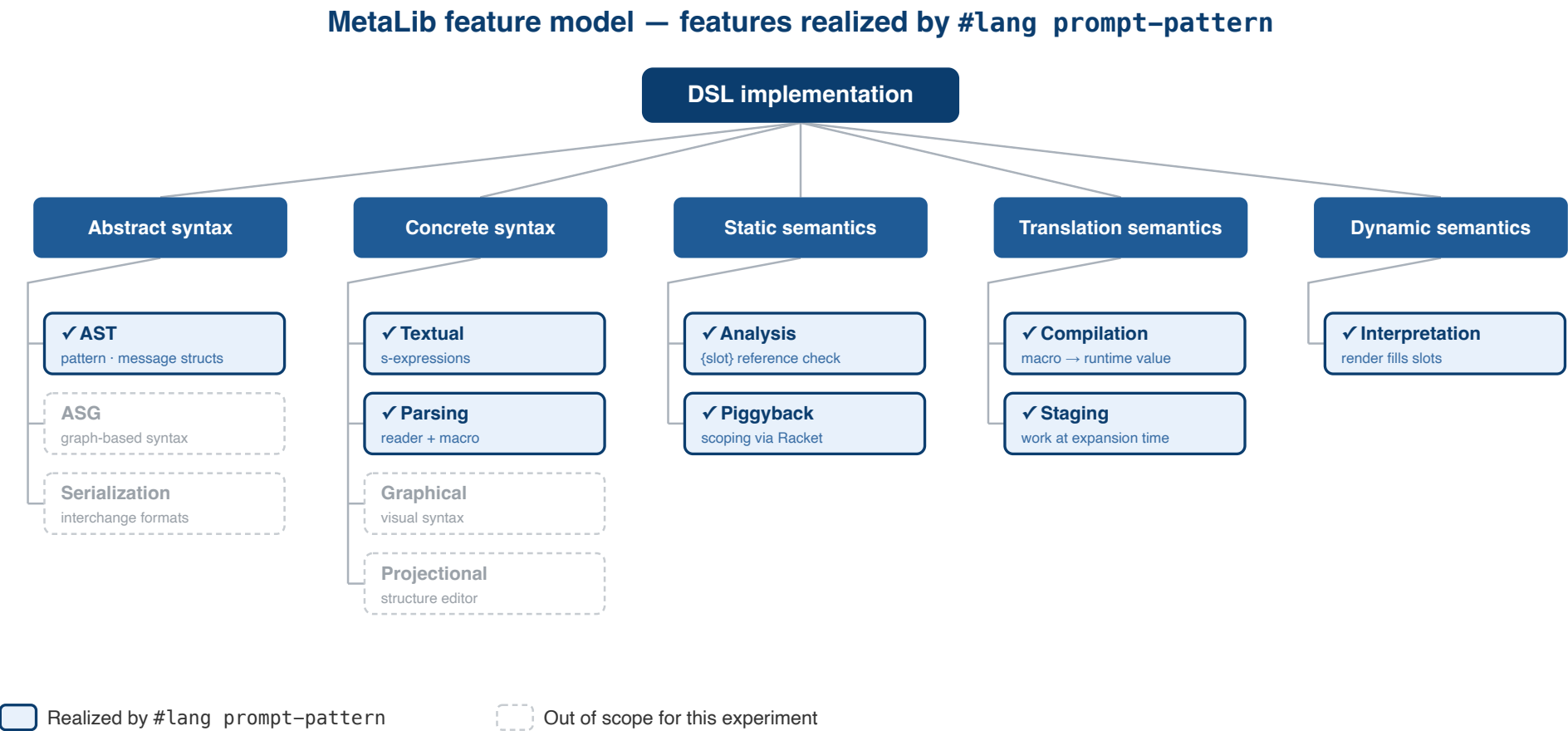
The program **fails to compile** (`make bad`):

```
examples/bad-pattern.rkt:9:2: define-pattern: slot {missing} is
referenced in a template but not declared in #:slots (name)
in: (user "Hello {name}, your account balance is {missing}.")
```

Raised by `raise-syntax-error` during macro expansion — and it points at the **user's** line, like a real compiler.

Live: the DrRacket Macro Stepper walks the `define-pattern` expansion.

Feature coverage — the MetaLib way



Every chrestomathy member is documented by which feature-model leaves
it realizes — Schauss & Lämmel et al., SLE 2017, Fig. 2.

Composition and interchange come for free

Chaining. Impromptu ships an explicit `chain` construct. In a *hosted* DSL a pattern is an ordinary value and `render` returns ordinary data — so feeding one prompt's result into the next is **plain Racket**, no new construct (`make chain`).

Interchange format. `render` returns a `jsexpr`, so the prompt is one `jsexpr→string` away from an API-ready request body:

```
[{"role":"system","content":"You are a translator. ..."},  
 {"role":"user","content":"Translate into German:\n..."}]
```

Both are lecture topics — composition, and interchange formats (JSON).

Where slot well-formedness is caught

Tool	Style	Slot check at
Impromptu (Langium)	external DSL	parse / validator time
DSPy (Python)	embedded library	run time
<code>#lang prompt-pattern</code> (Racket)	hosted DSL via macros	macro-expansion time

The chrestomathy frame **explains why this matters**: three tools occupy three points in the feature model and catch the *same* class of error at *different* stages of the lifecycle.

Generalising the methodology to N approaches yields a **PromptLib** in the spirit of MetaLib.

Reproduce it

GitHub: github.com/VivekanandaReddy3/sle-prompt-pattern

```
make install    # raco pkg install (idempotent)
make test       # 3 rackunit tests
make demo       # examples/greeting.rkt
make chain      # examples/chained.rkt – chaining + JSON
make bad        # examples/bad-pattern.rkt – compile-time error
```

Thank you. Questions?

References:

Schauss, Lämmel et al., *A Chrestomathy of DSL Implementations*, SLE 2017.

Clariso & Cabot, *Model-Driven Prompt Engineering*, MODELS 2023.

Khattab et al., *DSPy*, arXiv:2310.03714, 2023.